

React Frontend Framework Cheat Sheet

21 tables · 200 concepts · Updated 2026-07-16 · [cheatgrid.com](#)

React is a **declarative, component-based JavaScript library** for building user interfaces, maintained by Meta. It uses a **virtual DOM and Fiber reconciliation engine** to efficiently update only what changes in the actual DOM. React 19 — the most significant release since React 18 — introduced an action-based async model, Server Components, and new hooks (`useActionState`, `useFormStatus`, `useOptimistic`) that simplify data mutation and form handling. The **React Compiler** (v1.0, stable since October 2025) now automatically memoizes components at build time, making most manual `useMemo` and `useCallback` unnecessary in new code.

CONTENTS

Core Hooks

Component Fundamentals

JSX and Rendering

State Management

Effects and Lifecycle

Performance Optimization

Component Patterns

Event Handling

Forms and Validation

Refs and DOM Access

Context and Global State

Error Handling

Routing (React Router)

Testing

TypeScript Integration

Advanced React Features

Styling Approaches

DevTools and Debugging

Build Tools and Ecosystem

State Management Libraries

Accessibility Best Practices

References

1

3

3

4

5

5

6

7

7

8

9

9

10

11

12

12

13

14

14

15

16

16

TABLE 1

Core Hooks

Hooks are the functions that let a function component remember state, run side effects, and tap into React's machinery without a class. The ones here range from the everyday workhorses like `useState`, `useEffect`, and `useContext` to React 19's action-era additions like `useActionState` and `useOptimistic` that wire async work directly into your UI. If you only memorize one table on this sheet, make it this one.

Hook	Example	Description
<code>useState</code>	<code>const [count, setCount] = useState(0)</code>	• Adds local state to functional components • returns current value and updater function that triggers re-render.

Hook	Example	Description
useEffect	<pre>useEffect(() => { fetchData() }, [id])</pre>	- Runs side effects after render - empty array runs once on mount, deps trigger on changes, cleanup via return function.
useContext	<pre>const theme = useContext(ThemeContext)</pre>	- Consumes Context values without wrapping in Consumer - subscribes to nearest Provider above in tree.
useRef	<pre>const inputRef = useRef(null)</pre>	- Creates mutable reference that persists across renders without triggering updates - commonly used for DOM access.
useReducer	<pre>const [state, dispatch] = useReducer(reducer, init)</pre>	- Alternative to useState for complex state logic - accepts reducer <code>(state, action) => newState</code> following Redux pattern.
useMemo	<pre>const value = useMemo(() => compute(a, b), [a, b])</pre>	Memoizes computed values to avoid expensive recalculations - only recomputes when dependencies change.
useCallback	<pre>const handler = useCallback(() => fn(id), [id])</pre>	Memoizes function identity between renders - prevents child re-renders when passing callbacks as props.
useActionState	<pre>const [state, formAction, isPending] = useActionState(fn, init)</pre>	- React 19 hook that ties an async action to state - returns current state, a dispatchable action ref, and pending flag; eliminates manual loading/error <code>useState</code> boilerplate.
useFormStatus	<pre>const { pending } = useFormStatus()</pre>	- React 19 hook that reads the parent form's submission state - must be called from a component inside a <code><form></code> ; provides <code>pending</code> , <code>data</code> , <code>method</code> , <code>action</code> .
useOptimistic	<pre>const [optimistic, addOptimistic] = useOptimistic(state, fn)</pre>	- Shows optimistic UI update immediately before async action completes - automatically reverts on error.
useTransition	<pre>const [isPending, startTransition] = useTransition()</pre>	- Marks state updates as non-urgent transitions - React 19 supports async functions; keeps UI responsive by deferring or pausing low-priority work.
useDeferredValue	<pre>const deferredValue = useDeferredValue(searchQuery)</pre>	- Returns deferred version of value that lags behind during urgent updates - useful for expensive rendering like filtered lists.
useLayoutEffect	<pre>useLayoutEffect(() => { measure() }, [])</pre>	- Synchronous version of useEffect that fires before browser paint - use for DOM measurements to prevent visual flicker.
useId	<pre>const id = useId()</pre>	- Generates unique stable IDs for accessibility attributes - ensures matching IDs between server and client in SSR.
useImperativeHandle	<pre>useImperativeHandle(ref, () => ({ focus: () => input.focus() }))</pre>	- Customizes ref instance value exposed to parent - works with <code>forwardRef</code> or ref-as-prop (React 19+) to control imperative API.
useEffectEvent	<pre>const onMsg = useEffectEvent(data => { if (!muted) play(data) })</pre>	- Creates a non-reactive callback that always reads latest props/state - safe to call inside effects without adding to deps array; experimental in React 19.

TABLE 2

Component Fundamentals

Everything in React is built from components — small functions that take props and return JSX. These are the building blocks you compose all day: how data flows in through props, how children nest inside, and how `React.memo` , fragments, and React 19's ref-as-prop sharpen the basics.

Concept	Example	Description
Functional Component	<pre>function Button({ text }) { return <button>{text}</button> }</pre>	• Pure function that accepts props and returns JSX • preferred over class components since hooks in React 16.8.
Props	<pre><UserCard name="Alice" age={30} /></pre>	• Read-only data passed from parent to child • treated as function arguments enabling component reusability.
Children prop	<pre><Card> <h1>Title</h1> </Card></pre>	• Special prop containing nested JSX elements between opening and closing tags • enables composition patterns.
Default props	<pre>function Btn({ type = 'button' }) {}</pre>	• Fallback values for props using ES6 default parameters • replaced older <code>Component.defaultProps</code> static property.
Prop destructuring	<pre>function User({ name, email }) {}</pre>	• Extracts props directly in function parameters • supports nested destructuring and renaming.
Component composition	<pre><Layout> <Sidebar /> <Content /> </Layout></pre>	• Building complex UIs from simpler components • preferred over inheritance following "composition over inheritance" principle.
Pure component	<pre>const MemoComp = React.memo(Component)</pre>	• Component that returns same output for same props • <code>React.memo</code> wraps functional components for shallow prop comparison.
Fragment	<pre><> <td>A</td> <td>B</td> </></pre>	• Returns multiple elements without extra DOM nodes • shorthand <code><></code> or explicit <code><React.Fragment></code> with key support.
Ref as prop	<pre>function Input({ ref, ...props }) { return <input ref={ref} {...props} /> }</pre>	• React 19: ref can be passed as a regular prop to function components • eliminates need for <code>forwardRef</code> wrapper.

TABLE 3

JSX and Rendering

JSX is where JavaScript and markup meet. Curly braces drop expressions into your tree, and plain JavaScript handles the logic that templating languages bolt on with special syntax. These patterns cover the moves you reach for constantly: conditional rendering, mapping lists with stable keys, applying styles and classes, and escape hatches like portals and `dangerouslySetInnerHTML` .

Pattern	Example	Description
JSX expression	<pre><h1>Hello {user.name}</h1></pre>	• Embeds JavaScript expressions in markup using curly braces • evaluates to React elements at runtime.
Conditional rendering	<pre>{isLoggedIn ? <Dashboard /> : <Login />}</pre>	• Renders elements based on conditions using ternary operator, <code>&&</code> , or if statements • no special template syntax.
List rendering	<pre>{items.map(item => <Item key={item.id} {...item} />)}</pre>	• Transforms arrays into element lists using <code>map()</code> • requires unique <code>key</code> prop for reconciliation optimization.

Pattern	Example	Description
Key prop	<code><li key={user.id}>{user.name}</code>	- Helps React identify changed items in lists - should be stable and unique, avoid array indices for dynamic lists.
Document Metadata	<code><title>My Page</title> <meta name="description" content="..." /></code>	React 19: <code><title></code> , <code><link></code> , and <code><meta></code> tags rendered anywhere in the component tree are automatically hoisted to <code><head></code> .
Inline styling	<code><div style={{ color: 'red', fontSize: 16 }} /></code>	- Applies styles as JavaScript object with camelCased properties - numeric values auto-append <code>px</code> for lengths.
className	<code><div className="btn btn-primary" /></code>	Sets CSS classes using <code>className</code> (not <code>class</code>) to avoid the JavaScript reserved-word conflict.
dangerouslySetInnerHTML	<code><div dangerouslySetInnerHTML={{ __html: markup }} /></code>	- Injects raw HTML directly - named to highlight XSS risk, always sanitize untrusted content before use.
Portal	<code>createPortal(<Modal />, document.body)</code>	- Renders children into a DOM node outside the parent hierarchy - useful for modals and tooltips, events still bubble through the React tree.

TABLE 4

State Management

State is what makes a UI interactive, and most state bugs come from putting it in the wrong place. These techniques are the built-in toolkit before you reach for any library: local `useState`, controlled vs uncontrolled inputs, the Context API for sharing across the tree, and the judgment calls of lifting state up versus colocating it close to where it's used.

Technique	Example	Description
Local state	<code>const [name, setName] = useState('')</code>	- State scoped to single component using <code>useState</code> - re-renders component when updated.
State updater function	<code>setCount(prev => prev + 1)</code>	- Passes function to setter instead of value - ensures update uses most recent state, critical in async contexts.
Controlled component	<code><input value={text} onChange={e => setText(e.target.value)} /></code>	- Form element whose value is controlled by React state - value prop + onChange handler create single source of truth.
Context API	<code>const Ctx = createContext() <Ctx value={data}></code>	- Shares data across component tree without prop drilling - Provider supplies value, consumers subscribe via <code>useContext</code>.
Reducer pattern	<code>const reducer = (state, action) => { switch(action.type) {...} }</code>	- Centralizes complex state logic in pure reducer function - action objects describe state changes declaratively.
State batching	<code>setName('Alice') setAge(30)</code>	React 18+ groups multiple state updates into a single re-render automatically, even in promises and timeouts.
Derived state	<code>const fullName = firstName + ' ' + lastName</code>	- Values computed from props/state during render - avoid storing in separate state to prevent synchronization bugs.
Lifting state up	<code>const [val, setVal] = useState() <B onChange={setVal} /></code>	- Move state to nearest common ancestor when multiple siblings need the same data - passed down as props.
Uncontrolled component	<code><input defaultValue="hi" ref={inputRef} /></code>	- Form element that manages own state via DOM - access value through refs instead of state.
State colocation	<code>function Panel() { const [open, setOpen] = useState(false) }</code>	- Place state closest to where it's used - limits re-render scope and opposes premature lifting.

TABLE 5

Effects and Lifecycle

Effects are how a component stays in sync with the outside world — timers, subscriptions, event listeners, and data fetches that live beyond React's render. The recurring theme here is the cleanup function: anything you set up in an effect, you tear down before the next run or on unmount, or you leak memory and stale listeners.

Pattern	Example	Description
Effect with cleanup	<pre>useEffect(() => { const id = setInterval(tick, 1000) return () => clearInterval(id) }, [])</pre>	• Effect returning cleanup function runs before unmount or before next effect • prevents memory leaks.
Dependency array	<pre>useEffect(() => fetch(url), [url])</pre>	Controls when effect runs — empty array runs once on mount, omitted runs every render, with deps runs on changes.
Custom Hook	<pre>function useWindowSize() { const [size, setSize] = useState(...) useEffect(() => {...}, []) return size }</pre>	• Extracts reusable stateful logic into function prefixed with "use" • can call other hooks internally.
Effect synchronization	<pre>useEffect(() => { socket.connect() return () => socket.disconnect() }, [roomId])</pre>	• Keeps component synchronized with external system • connection logic mirrors component lifecycle.
Data fetching	<pre>useEffect(() => { fetch(url).then(r => r.json()).then(setData) }, [url])</pre>	• Loading external data in effect • consider React Query or SWR for production use to handle race conditions.
Event listeners	<pre>useEffect(() => { window.addEventListener('resize', handle) return () => window.removeEventListener('resize', handle) }, [])</pre>	• Attaching global event handlers outside React's event system • cleanup prevents duplicate listeners.
Interval/Timeout	<pre>useEffect(() => { const id = setTimeout(() => setShow(true), 1000) return () => clearTimeout(id) }, [])</pre>	• Running timed operations • cleanup cancels pending timers to avoid state updates on unmounted components.
Layout effect	<pre>useLayoutEffect(() => { const h = ref.current.offsetHeight setHeight(h) }, [])</pre>	• Fires synchronously before browser paint • use for DOM measurements that affect layout to prevent flicker.

TABLE 6

Performance Optimization

Most React performance work comes down to rendering less and rendering smaller. With the React Compiler now stable, much of the old manual memoization is handled for you, but knowing what `React.memo`, code splitting, list virtualization, and the concurrent hooks actually do still tells you where to look when a UI feels sluggish.

Technique	Example	Description
React Compiler	<pre>// babel-plugin-react-compiler // no code changes required</pre>	• Build-time tool (v1.0, stable Oct 2025) that automatically memoizes components and hooks • makes most manual <code>useMemo</code> / <code>useCallback</code> unnecessary; opt-in via Babel/Vite plugin.
React.memo	<pre>const List = React.memo(function List({ items }) {...})</pre>	• Prevents re-renders if props haven't changed using shallow comparison • second argument allows custom comparison.

Technique	Example	Description
useMemo	<pre>const sorted = useMemo(() => items.sort(), [items])</pre>	• Caches expensive computations between renders • React Compiler handles this automatically when enabled.
useCallback	<pre>const onClick = useCallback(() => fn(id), [id])</pre>	• Memoizes function reference to prevent unnecessary child re-renders • React Compiler handles this automatically when enabled.
Code splitting	<pre>const Comp = React.lazy(() => import('./Comp'))</pre>	• Splits bundle into smaller chunks loaded on demand • <code>React.lazy</code> with dynamic <code>import()</code> creates split points.
Suspense	<pre><Suspense fallback={<Spinner />}> <LazyComp /> </Suspense></pre>	• Shows fallback UI while lazy component or async data loads • works with <code>React.lazy</code> and data fetching libraries.
List virtualization	<pre><FixedSizeList height={400} itemCount={1000} itemSize={35}></pre>	• Renders only visible list items using libraries like <code>react-window</code> • dramatically improves performance for long lists.
useTransition	<pre>startTransition(() => setQuery(input))</pre>	• Marks updates as low priority • keeps UI responsive during expensive re-renders by allowing React to interrupt work.
useDeferredValue	<pre>const deferred = useDeferredValue(searchTerm)</pre>	• Creates laggy version of value that updates after urgent renders • alternative to debouncing for responsive UIs.
Key optimization	<pre>items.map(item => <Item key={item.id} />)</pre>	• Using stable unique keys helps React identify which items changed • avoids unnecessary DOM mutations.
Production build	<pre>npm run build</pre>	• Creates optimized bundle with minification and dead code elimination • significantly faster than development build.

TABLE 7

Component Patterns

These are the reusable shapes experienced React developers reach for to keep components flexible and DRY. Some of them, like compound components, the provider pattern, and composition through children, are alive and well. Others like render props and higher-order components have largely been replaced by custom hooks, so this table doubles as a map of which patterns still earn their keep.

Pattern	Example	Description
Compound components	<pre><Tabs> <Tabs.List> <Tabs.Tab /> </Tabs.List> </Tabs></pre>	• Components that work together sharing implicit state • creates a cohesive API like native HTML <code><select></code> and <code><option></code> .
Provider pattern	<pre><ThemeProvider> <App /> </ThemeProvider></pre>	• Wraps tree with Context Provider to share values • avoids prop drilling for global concerns like theme or auth.
Render props	<pre><DataProvider render={data => <Display data={data} /> /></pre>	• Passes a function as a prop that the component calls to determine render output • largely superseded by custom hooks.
Higher-Order Component	<pre>const Enhanced = withAuth(Component)</pre>	• Function that takes a component and returns an enhanced version • adds props or wraps with logic. Less common since hooks and the React Compiler.
Composition	<pre><Card> {children} </Card></pre>	• Building components by combining smaller ones • uses the <code>children</code> prop to accept nested content flexibly.
Slot pattern	<pre><Layout header={<Header />} sidebar={<Sidebar />} /></pre>	• Passes multiple JSX elements as named props • more explicit than <code>children</code> for complex layouts.

Pattern	Example	Description
Container/Presentational	<pre>// Container: fetches data // Presentational: renders UI</pre>	• Separates logic from presentation • containers handle state and effects, presentational components are pure render functions.
Controlled vs Uncontrolled	<pre>// Controlled: value={state} // Uncontrolled: ref={inputRef}</pre>	• Controlled components use React state as source of truth • uncontrolled rely on DOM state accessed via refs.
Activity component	<pre><Activity mode="hidden"> <ExpensivePanel /> </Activity></pre>	• React 19.2: hides or shows UI while preserving component state and DOM • its Effects are cleaned up while hidden. <code>mode</code> is <code>"visible"</code> or <code>"hidden"</code> .

TABLE 8

Event Handling

React wraps native browser events in a cross-browser SyntheticEvent and attaches a single listener at the root rather than wiring up handlers on every node. The entries here cover the everyday mechanics: passing handlers, reading the event object, binding extra arguments, and the inline-versus-named-handler tradeoff.

Concept	Example	Description
Event handler	<pre><button onClick={handleClick}>Click</button></pre>	• Function passed to event prop • convention names handlers <code>handleEvent</code> or <code>onEvent</code> .
Event object	<pre>function handle(e) { e.preventDefault() console.log(e.target.value) }</pre>	• SyntheticEvent instance with <code>target</code> , <code>currentTarget</code> • methods <code>preventDefault()</code> , <code>stopPropagation()</code> .
Synthetic events	<pre>onClick={e => console.log(e.type)}</pre>	• React's cross-browser wrapper around native events • normalized API works consistently across all browsers.
Passing arguments	<pre>onClick={() => deleteRow(id)}</pre>	• Binding additional parameters to handlers using arrow function or <code>bind</code> • arrow function is more common.
Named handler	<pre>const handleClick = () => {...} <button onClick={handleClick}></pre>	• Declaring function separately then passing reference • better for reusability and testing than inline.
Inline handler	<pre><button onClick={() => alert('clicked')}></pre>	• Defining function directly in JSX • convenient for simple logic but creates new function each render.
Event delegation	React attaches all handlers to the root container ; events bubble up to a single listener for performance.	• React 17+ attaches to root instead of document • improves micro-frontend isolation and compatibility.
Custom events	<pre>const event = new CustomEvent('myEvent') element.dispatchEvent(event)</pre>	• Dispatching browser custom events • React handlers work with native custom events on refs.

TABLE 9

Forms and Validation

Forms are where state, events, and validation all collide. The classic React approach is the controlled input, its value driven by state and updated on every change, but React 19's form actions now hand pending state, errors, and resets back to the framework. This table walks each control type plus when a library like React Hook Form is worth the dependency.

Technique	Example	Description
Controlled input	<pre><input value={name} onChange={e => setName(e.target.value)} /></pre>	• Input value controlled by state • onChange handler updates state creating bidirectional binding.

Technique	Example	Description
Form Actions	<pre>const [state, formAction] = useActionState(fn, init) <form action={formAction}></pre>	- React 19: native <code>action</code> prop accepts an async function - React manages pending state, errors, and resets automatically.
Form submission	<pre><form onSubmit={e => { e.preventDefault() submit(data) }}></pre>	Intercepting submit event to prevent default page reload and handle data programmatically.
Multiple inputs	<pre>const change = e => setState({...state, [e.target.name]: e.target.value})</pre>	Handling multiple form fields with single handler using <code>name</code> attribute and computed property names.
Checkbox	<pre><input type="checkbox" checked={agreed} onChange={e => setAgreed(e.target.checked)} /></pre>	- Uses <code>checked</code> prop and <code>e.target.checked</code> for value - controlled with boolean state.
Select dropdown	<pre><select value={selected} onChange={e => setSelected(e.target.value)}> <option value="a">A</option> </select></pre>	- Uses <code>value</code> on <code><select></code> not <code>selected</code> on <code><option></code> single source of truth pattern.
Textarea	<pre><textarea value={text} onChange={e => setText(e.target.value)} /></pre>	- Uses <code>value</code> prop like inputs (not children) - controlled the same way for consistency.
File input	<pre><input type="file" ref={fileRef} /></pre>	- Always uncontrolled because value is read-only - access files via <code>fileRef.current.files</code>.
Form libraries	<pre>const { register, handleSubmit } = useForm()</pre>	Libraries like React Hook Form or Formik reduce boilerplate for validation, submission, and field management.
Validation	<pre>const error = email && !email.includes('@') ? 'Invalid email' : null</pre>	- Implementing inline validation by checking state values - display errors conditionally based on validation rules.

TABLE 10

Refs and DOM Access

Refs are React's escape hatch to the actual DOM and to mutable values that shouldn't trigger re-renders. Reach for them when you need to focus an input, measure a node, or hold a value like a timer ID across renders. React 19 simplified the whole area: `ref` is now a regular prop and `ref` callbacks can return cleanup functions, retiring the old `forwardRef` ceremony.

Method	Example	Description
useRef	<pre>const inputRef = useRef(null) <input ref={inputRef} /></pre>	- Creates mutable reference to DOM element or value <code>current</code> persists without causing re-renders.
DOM manipulation	<pre>inputRef.current.focus()</pre>	- Directly calling DOM methods via <code>ref</code> - escape hatch for imperative operations React doesn't provide.
Ref as prop	<pre>function Input({ ref }) { return <input ref={ref} /> }</pre>	- React 19: ref is a regular prop in function components - no <code>forwardRef</code> wrapper needed, simpler and cleaner.
Ref forwarding	<pre>const Comp = forwardRef((props, ref) => <input ref={ref} />)</pre>	- Legacy pattern passing <code>ref</code> from parent through component to child - still works but <code>forwardRef</code> will be deprecated in favor of <code>ref-as-prop</code>.
Ref cleanup	<pre><div ref={node => { setup(node) return () => cleanup(node) }} /></pre>	- React 19: <code>ref</code> callbacks can return a cleanup function called when element unmounts - replaces manual <code>null</code> check pattern.
Callback ref	<pre><div ref={node => setNode(node)} /></pre>	- Ref as function called with DOM node on mount and <code>null</code> on unmount - useful for measuring or observing nodes.

Method	Example	Description
Refs in loops	<pre>const itemRefs = useRef(new Map()) <div ref={node => itemRefs.current.set(id, node)} /></pre>	- Storing multiple refs in Map or array - ref callback pattern manages dynamic lists of refs.
useImperativeHandle	<pre>useImperativeHandle(ref, () => ({ focus: () => inputRef.current.focus() }))</pre>	- Customizes instance value exposed to parent via ref - controls what methods/properties are public.
Storing values	<pre>const countRef = useRef(0) countRef.current++</pre>	- Using ref to hold mutable value that doesn't trigger re-renders - useful for intervals, previous values, instance variables.

TABLE 11

Context and Global State

Context solves prop drilling — passing data straight to deep consumers without threading it through every layer in between. These entries take you from `createContext` to the Context-plus- `useReducer` combo that serves as a lightweight Redux for simpler apps, and the memoization tricks that stop a Provider from re-rendering every consumer on each change.

Pattern	Example	Description
createContext	<pre>const ThemeContext = createContext('light')</pre>	- Creates Context object with default value - default used when no Provider exists above in tree.
Context Provider	<pre><ThemeContext value="dark"> <App /> </ThemeContext></pre>	React 19: use context directly as a Provider — <code><ThemeContext value="..."></code> replaces legacy <code><ThemeContext.Provider value="..."></code> .
useContext	<pre>const theme = useContext(ThemeContext)</pre>	- Reads context value from nearest Provider above - replaces older <code>Context.Consumer</code> pattern.
Context with reducer	<pre>const [state, dispatch] = useReducer(reducer, init) <Ctx value={{state, dispatch}}></pre>	- Combining Context + useReducer for global state management - lightweight alternative to Redux for simpler apps.
Custom provider	<pre>function DataProvider({ children }) { const [data, setData] = useState() return <Ctx value={{data, setData}}>{children}</Ctx> }</pre>	- Wrapping Provider in component with logic - encapsulates state management and provides clean API.
Multiple contexts	<pre><ThemeProvider> <UserProvider> <App /> </UserProvider> </ThemeProvider></pre>	- Nesting multiple Providers for different concerns - component can consume multiple contexts with separate <code>useContext</code> calls.
Context performance	<pre>const value = useMemo(() => ({state, dispatch}), [state])</pre>	Memoizing context value prevents unnecessary consumer re-renders - split contexts to limit update scope.

TABLE 12

Error Handling

A thrown error during render shouldn't take down the whole app — error boundaries catch it and show a fallback instead. The key gotcha worth internalizing: boundaries only catch errors in rendering and lifecycle, not in event handlers or async code, so those still need plain try-catch.

Technique	Example	Description
Error Boundary	<pre>class ErrorBoundary extends React.Component { componentDidCatch(error, info) {...} render() { return this.state.hasError ? <Fallback /> : this.props.children } }</pre>	• Class component that catches JavaScript errors in child tree • displays fallback UI instead of crashing the entire app.
getDerivedStateFromError	<pre>static getDerivedStateFromError(error) { return { hasError: true } }</pre>	• Lifecycle method for updating state when error is thrown • used to render fallback in the next render pass.
componentDidCatch	<pre>componentDidCatch(error, errorInfo) { logError(error, errorInfo.componentStack) }</pre>	• Lifecycle for side effects after error is caught • log errors to reporting service while displaying fallback.
Error boundary library	<pre><ErrorBoundary FallbackComponent={ErrorFallback}> <App /> </ErrorBoundary></pre>	react-error-boundary provides hooks-based error handling (<code>useErrorBoundary</code>), reset, and retry capabilities.
Suspense boundary	<pre><Suspense fallback={<Loading />}> <AsyncComp /> </Suspense></pre>	• Shows fallback while async components or data loads • catches thrown promises from <code>React.lazy</code> or data fetching.
Try-catch in handlers	<pre>try { riskyOperation() } catch (error) { setError(error.message) }</pre>	• Manual error handling in event handlers or effects • Error Boundaries do not catch errors in these contexts.

TABLE 13

Routing (React Router)

React ships no router of its own, so React Router is the de facto standard for mapping URLs to components in single-page apps. This covers the core pieces, routers, route definitions, navigation links, and the hooks that read params and location, plus a nod to TanStack Router for teams that want fully type-safe routing.

Component	Example	Description
BrowserRouter	<pre><BrowserRouter> <App /> </BrowserRouter></pre>	• Uses the HTML5 History API for clean URLs • wraps app to enable client-side routing without a hash.
Routes	<pre><Routes> <Route path="/" element={<Home />} /> </Routes></pre>	• Container for route definitions replacing older <code><Switch></code> • renders the single best match, not just the first.
Route	<pre><Route path="/users/:id" element={<User />} /></pre>	• Defines path-to-component mapping • dynamic segments start with <code>:</code> for parameters.
Link	<pre><Link to="/about">About</Link></pre>	• Creates navigation links without a full page reload • renders an anchor tag with a client-side click handler.
NavLink	<pre><NavLink to="/home" className={({isActive}) => isActive ? 'active' : '' /></pre>	• Like Link but adds active styling when the current route matches • useful for navigation menus.
useNavigate	<pre>const navigate = useNavigate() navigate('/dashboard')</pre>	• Hook for programmatic navigation • replaces the older <code>useHistory</code> with a simpler API.
useParams	<pre>const { id } = useParams()</pre>	• Extracts URL parameters from the current route • returns an object of string values keyed by param name.

Component	Example	Description
useLocation	<pre>const location = useLocation()</pre>	- Returns the current location object with <code>pathname</code>, <code>search</code>, <code>hash</code>, <code>state</code> - useful for tracking page views.
Nested routes	<pre><Route path="/users" element={<Users />}> <Route path=:id" element={<User /> } /> </Route></pre>	- Defines child routes under a parent - relative child paths combine with the parent path.
Outlet	<pre>function Layout() { return <div><Nav /><Outlet /></div> }</pre>	- Renders the matched child route inside the parent - enables nested layouts.
TanStack Router	<pre>const route = createRoute({ getParentRoute: () => rootRoute, path: '/users/\$id', })</pre>	- Fully type-safe alternative to React Router - search params, path params, and loader data are all TypeScript-inferred with no manual annotations.

TABLE 14

Testing

The modern React testing philosophy is to test what the user does, not how the component is wired internally. React Testing Library leans hard into that by querying through accessible roles and text. Pair it with a runner like Jest or Vitest and the helpers here, and your tests survive refactors instead of breaking on them.

Tool	Example	Description
React Testing Library	<pre>render(<Button />) screen.getByRole('button')</pre>	- Encourages testing user behavior not implementation - queries based on accessibility attributes.
userEvent	<pre>await user.click(button)</pre>	- More realistic user interaction simulation than <code>fireEvent</code> - handles focus, timing, and keyboard events properly.
screen queries	<pre>screen.getByText('Submit')</pre>	Built-in query methods like <code>getBy</code> , <code>queryBy</code> , <code>findBy</code> with variants for text, role, label, test ID, etc.
waitFor	<pre>await waitFor(() => expect(screen.getByText('Loaded')).toBeInTheDocument())</pre>	- Waits for async changes like state updates or API responses - retries until callback stops throwing.
render	<pre>const { container } = render(<App />)</pre>	- Renders component into virtual DOM for testing - returns queries and container element.
fireEvent	<pre>fireEvent.click(button)</pre>	- Triggers DOM events on elements - synchronous simulation — prefer <code>userEvent</code> for realistic interaction.
Jest	<pre>test('renders title', () => {...})</pre>	Test runner and assertion library - provides <code>describe</code>, <code>test</code>, <code>expect</code>, and mocking capabilities.
Vitest	<pre>import { test, expect } from 'vitest' test('adds', () => expect(1+1).toBe(2))</pre>	Jest-compatible test runner built on Vite - significantly faster cold starts and watch mode; ideal for Vite-based React projects; easy migration path from Jest (swap <code>jest.fn</code> for <code>vi.fn</code>; globals are off by default).
Mock functions	<pre>const onClick = jest.fn() expect(onClick).toHaveBeenCalledTimes(1)</pre>	- Creates spy functions to verify calls <code>jest.fn()</code> (or <code>vi.fn()</code> in Vitest) tracks arguments and call count.
Snapshot testing	<pre>expect(tree).toMatchSnapshot()</pre>	- Captures component output and detects unexpected changes - useful for catching regressions in UI structure.

TABLE 15

TypeScript Integration

TypeScript and React are a near-default pairing now, and most of the friction is in typing the few things TypeScript can't infer on its own. These patterns cover the spots that trip people up: typing props, state generics, event handlers, refs, and children, small habits that catch a surprising share of bugs before the code ever runs.

Pattern	Example	Description
Props interface	<pre>interface Props { name: string; age: number } function User(props: Props) {}</pre>	- Separate type definition for props - enables reuse and clearer documentation.
Component typing	<pre>function Button({ text }: { text: string }) {}</pre>	- Inline props type for simple components - clear and concise for few props.
useState typing	<pre>const [user, setUser] = useState<User null>(null)</pre>	- Explicitly typing state generic when TypeScript can't infer - necessary for complex types or null initialization.
Event typing	<pre>const handle = (e: React.ChangeEvent<HTMLInputElement>) => {}</pre>	- Typing event handlers with specific event types - ensures type safety for <code>e.target</code> properties.
Ref typing	<pre>const ref = useRef<HTMLDivElement>(null)</pre>	- Providing element type for ref <code>null</code> initial value requires union type for <code>.current</code>.
Children typing	<pre>interface Props { children: React.ReactNode }</pre>	- Using ReactNode type for children prop - accepts any valid React child (elements, strings, numbers, fragments).
Generic components	<pre>function List<T>(props: { items: T[]; render: (item: T) => ReactNode }) {}</pre>	- Components accepting generic type parameters - enables type-safe reusable components.
Context typing	<pre>const Ctx = createContext<ContextType undefined>(undefined)</pre>	- Typing Context value and initial value <code>undefined</code> forces Provider usage with runtime check.
FC type	<pre>const Button: React.FC<Props> = ({ text }) => {}</pre>	- Typing component as FunctionComponent - modern React types dropped its implicit <code>children</code>, so it is now mostly a stylistic choice.

TABLE 16

Advanced React Features

This is where React stops being "just a UI library" and becomes a full-stack rendering architecture. Concurrent rendering, Server Components, Server Actions, the React Compiler, and streaming SSR are the modern frontier, features that shift work to the server, ship less JavaScript, and keep the UI responsive under load.

Feature	Example	Description
Concurrent rendering	<pre>createRoot(el).render(<App />)</pre>	- React 18+ feature enabling interruptible rendering - allows React to pause, abort, and prioritize urgent updates.
Automatic batching	<pre>setName('Alice') setAge(30)</pre>	React 18+ groups multiple state updates into a single re-render, even in promises and timeouts.
React Compiler	<pre>// install: babel-plugin-react-compiler // or: vite-plugin-react with babel option</pre>	- Build-time optimizer (v1.0 stable, Oct 2025) that automatically memoizes components - eliminates need for manual <code>useMemo</code> / <code>useCallback</code>.
Server Components	<pre>async function Page() { const data = await fetchData() return <Product data={data} /> }</pre>	- Async components that run only on server - fetch data directly, render to HTML, send zero JS for that part to client.

Feature	Example	Description
'use client' directive	<pre>'use client' import { useState } from 'react'</pre>	• Marks a module boundary for client-side code in RSC • components below this point ship JS to the browser.
Server Actions	<pre>'use server' async function submitForm(formData) { await db.save(formData) }</pre>	• Functions marked with 'use server' that run server-side • called from client components or form action props.
Activity component	<pre><Activity mode="hidden"> <ExpensivePanel /> </Activity></pre>	• React 19.2: hides or shows UI without unmounting • preserves state and DOM, but destroys Effects while hidden and re-creates them when visible again • mode is "visible" or "hidden" .
ViewTransition	<pre><ViewTransition> {url === '/' ? <Home /> : <Details />} </ViewTransition></pre>	Experimental: wraps elements to enable browser-native View Transition API animations on startTransition , useDeferredValue , or Suspense changes.
Streaming SSR	<pre>renderToPipeableStream(<App />)</pre>	• Server-side rendering that streams HTML to client progressively • sends ready content first using Suspense boundaries.
startTransition	<pre>startTransition(async () => { await submitData(form) })</pre>	• Marks updates as non-urgent transitions • React 19 supports async functions, keeping UI responsive without blocking.
Strict Mode	<pre><StrictMode> <App /> </StrictMode></pre>	• Development-only tool that highlights potential problems • intentionally double-invokes functions to surface side effects.
Profiler	<pre><Profiler id="Nav" onRender={callback}> <Nav /> </Profiler></pre>	• Measures rendering performance programmatically via an onRender callback • profiling is disabled in production builds by default.
use hook	<pre>const data = use(promise)</pre>	• React 19 API that reads resources like promises or context • not a Hook, so uniquely it can be called conditionally unlike other hooks.

TABLE 17

Styling Approaches

There's no one blessed way to style React, and the options here represent genuinely different philosophies: utility classes with Tailwind, locally scoped CSS Modules, CSS-in-JS libraries like Styled Components and Emotion, preprocessors like Sass, and plain inline styles. Each makes its own tradeoff between speed, scoping, and runtime cost.

Method	Example	Description
Tailwind CSS	<pre><div className="flex items-center gap-4 p-4 bg-white" /></pre>	• Utility-first framework: compose predefined single-purpose classes in the markup • rapid development, and one of the most widely adopted React styling choices today.
CSS Modules	<pre>import styles from './Button.module.css' <div className={styles.button} /></pre>	• Locally scoped CSS by default • build tool hashes class names to unique identifiers to prevent global conflicts.
Styled Components	<pre>const Button = styled.button`color: blue;`</pre>	• CSS-in-JS library using tagged template literals • styles scoped to the component, with prop-based dynamic styling and theming.
Emotion	<pre><div css={css`color: red;`} /></pre>	• CSS-in-JS supporting both string and object styles • adds a css prop for direct styling, widely used in design systems.

Method	Example	Description
Sass/SCSS	<code>import './styles.scss'</code>	• CSS preprocessor with variables, nesting, mixins • must be compiled to plain CSS at build time; browsers can't read SCSS directly.
Inline styles	<code><div style={{ backgroundColor: 'blue', padding: 20 }} /></code>	• JavaScript object with camelCased properties; a plain number gets px appended • no pseudo-classes or media queries, best for dynamic one-off values.
CSS-in-JS	Libraries embedding styles in JavaScript; component-scoped with access to props and state for dynamic styling.	• Generic term for styling solutions collocating CSS with components • runtime variants inject styles per render; zero-runtime options extract CSS at build.

TABLE 18

DevTools and Debugging

When something renders wrong or too often, these are the tools that show you why. React DevTools lets you walk the live component tree and inspect props, state, and hooks; the Profiler tells you what re-rendered and how long it took; and Strict Mode surfaces hidden bugs by deliberately double-invoking your components in development.

Tool	Example	Description
React DevTools	Browser extension showing component tree with props, state, hooks; inspect and edit values live.	• Official debugging browser extension for Chrome and Firefox • essential for development.
Components tab	Navigate component hierarchy ; view props, state, hooks, owners, source code location.	• Explores React component structure in DevTools • search and filter components.
Profiler tab	Records rendering performance ; flame graph shows which components render and why.	• Identifies performance bottlenecks • measures commit phases and component render time.
Console logging	<code>console.log('State:', state)</code>	• Basic debugging by printing values • effective for understanding data flow and spotting unexpected updates.
React strict mode warnings	Logs warnings about deprecated APIs and unsafe lifecycles; double-invokes effects to surface hidden bugs.	• Highlights potential problems in development • identifies impure components and stale closures.
Source maps	<code>devtool: 'source-map'</code> in config	• Maps compiled code to source • enables debugging original TypeScript/JSX in browser despite minification.
Error boundaries	Catch and log render errors ; prevent entire app crash with fallback UI.	Provides graceful error handling in component tree.
React Suspense DevTools	DevTools show Suspense boundaries and loading states; visualizes loading sequences and boundary states.	Helps debug async loading sequences and Suspense tree structure.

TABLE 19

Build Tools and Ecosystem

Picking how to start a React project now matters as much as the code itself. This is the lay of the land in 2026: Vite for fast SPAs, Next.js as the dominant production framework, newer entrants like TanStack Start, plus the bundlers and transpilers underneath. Create React App is here mainly as a signpost: it is no longer maintained, so reach for Vite or Next instead.

Tool	Example	Description
Vite	<code>npm create vite@latest my-app -- --template react</code>	• Fast dev server serving your source over native ES modules • bundles production builds with Rollup (now the Rust-based Rolldown), not webpack • recommended for new projects.

Tool	Example	Description
Next.js	<code>npx create-next-app@latest</code>	• Production framework with SSR, SSG, routing, API routes, and React Server Components built-in • most popular production React framework.
TanStack Start	<code>npx create-tsrouter-app@latest my-app</code>	• Full-stack framework built on TanStack Router • type-safe routing, server functions, and SSR • rapidly growing alternative to Next.js.
Remix	<code>npx create-remix@latest</code>	• Framework focused on web fundamentals • nested routing, progressive enhancement, optimistic UI • merged into React Router v7.
React Native	<code>npx react-native init MyApp</code>	• Build native mobile apps using React • JavaScript drives real native platform UI, shared across iOS and Android with platform-specific code.
Gatsby	<code>npm init gatsby</code>	• Static site generator with GraphQL data layer • builds HTML pages at build time, excellent for content-heavy sites and blogs.
Webpack	Module bundler that compiles JavaScript and assets; highly configurable but complex setup.	• Flexible build tool used by legacy React projects • builds a dependency graph and combines modules into bundles.
Turbopack	Built into Next.js, run via <code>next dev</code>	• Rust-based incremental bundler by Vercel • built into Next.js as its default bundler • significantly faster cold starts and incremental builds.
Babel	<code>@babel/preset-react</code>	• Transpiles JSX and modern JS to compatible JavaScript • does not bundle modules, that is a separate job.
ESLint	<code>eslint-plugin-react</code> , <code>eslint-plugin-react-hooks</code>	• Linting tool that statically analyzes code to find problems • React-specific plugins enforce best practices and hooks rules.
TypeScript	<code>npm create vite@latest my-app -- --template react-ts</code>	• Adds static typing to JavaScript, checked at compile time then erased • catches errors before runtime, improves IDE support • recommended for all new projects.
Create React App (legacy)	<code>npx create-react-app my-app</code>	• Zero-config starter, no longer actively maintained • use Vite or Next.js for new projects.

TABLE 20

State Management Libraries

When `useState` and Context stop scaling, these libraries take over, and they don't all solve the same problem. Redux Toolkit, Zustand, and Jotai handle client state with very different ergonomics, while TanStack Query and SWR specialize in server state: caching, syncing, and revalidating data from APIs. Knowing which bucket your problem falls into is half the decision.

Library	Example	Description
Redux Toolkit	<code>const store = configureStore({ reducer })</code> <code>useSelector(state => state.user)</code>	• Modern Redux with less boilerplate • includes <code>createSlice</code> , <code>createAsyncThunk</code> , opinionated defaults • best for large teams.
Zustand	<code>const useStore = create(set => ({ count: 0, inc: () => set(s => ({ count: s.count + 1 })))</code>	• Minimal state management with hooks-based API • no providers, boilerplate-free, tiny bundle • most popular lightweight option.
TanStack Query	<code>const { data } = useQuery({ queryKey: ['user'], queryFn: fetchUser })</code>	• Server state management with caching, sync, and background updates • not general state but the standard for async data.
SWR	<code>const { data } = useSWR('/api/user', fetcher)</code>	• Data fetching hook with caching • "stale-while-revalidate" strategy • lightweight Vercel-maintained alternative to TanStack Query.
Jotai	<code>const [count, setCount] = useAtom(countAtom)</code>	• Primitive and flexible atom-based state • bottom-up approach, no string keys, TypeScript-first.

Library	Example	Description
MobX	<pre>const store = observable({ count: 0 }) const Counter = observer(() => <div>{store.count}</div>)</pre>	• Reactive state management using observables • automatic dependency tracking, minimal boilerplate.
Recoil	<pre>const [text, setText] = useRecoilState(textState)</pre>	• Facebook's atomic state library with fine-grained subscriptions • not actively maintained (GitHub repo archived Jan 2025), prefer Jotai for similar patterns.

TABLE 21

Accessibility Best Practices

Accessibility isn't a feature you bolt on at the end. It starts with using the right HTML element for the job, so a button is a `<button>` and keyboard support comes for free. These practices cover the rest: labelling inputs, managing focus, the ARIA attributes to use when native semantics fall short, and the contrast ratios that keep content readable for everyone.

Practice	Example	Description
Semantic HTML	<pre><button>Click</button> not <div onClick>Click</div></pre>	• Use correct HTML elements for functionality • provides built-in keyboard support and screen reader context.
Form labels	<pre><label htmlFor="email">Email</label> <input id="email" /></pre>	• Associate labels with inputs using <code>htmlFor</code> • enables clicking label to focus input • required for screen readers.
Focus management	<pre>useEffect(() => inputRef.current.focus(), [])</pre>	• Programmatically move focus to appropriate elements • essential for modals, route changes, dynamic content.
ARIA attributes	<pre><div role="button" aria-label="Close" /></pre>	• Add accessibility semantics when semantic HTML is insufficient • use sparingly, and prefer native elements.
aria-label	<pre><button aria-label="Close dialog">x</button></pre>	Provides text alternative for screen readers when visible text is insufficient or absent.
Live regions	<pre><div aria-live="polite" aria-atomic="true">{status}</div></pre>	• Announce dynamic content changes to screen readers • <code>polite</code> waits for pause, <code>assertive</code> interrupts.
Keyboard navigation	<pre>onKeyDown={e => e.key === 'Enter' && handleClick() }</pre>	• Ensure all interactive elements are accessible via keyboard • implement arrow keys for custom widgets.
useld for a11y	<pre>const id = useId() <label htmlFor={id}>Name</label> <input id={id} /></pre>	• Generates stable unique IDs for linking labels and inputs • safe in SSR, so it avoids hydration mismatches.
Skip links	<pre>Skip to main</pre>	• Bypass navigation links at page top • allows keyboard users to jump directly to content.
Alt text	<pre></pre>	• Describe image content for screen readers • use empty <code>alt=""</code> for purely decorative images.
Color contrast	Verify with tools like axe or Lighthouse	Minimum 4.5:1 ratio for normal text, 3:1 for large text per WCAG 2.1 AA.

References

React & web framework docs

1. 'use client' directive - <https://react.dev/reference/rsc/use-client>
 3. ARIA attributes - <https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA>
 5. BrowserRouter - <https://reactrouter.com/api/declarative-routers/BrowserRouter>
 7. Checkbox - <https://react.dev/reference/react-dom/components/input#my-checkbox-doesnt-update-when-i-click-on-it>
2. Activity component - <https://react.dev/reference/react/Activity>
 4. aria-label - <https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Attributes/aria-label>
 6. Callback ref - <https://react.dev/reference/react-dom/components/common#ref-callback>
 8. Children prop - <https://react.dev/learn/passing-props-to-a-component#passing-jsx-as-children>

9. Code splitting - <https://react.dev/reference/react/lazy>
11. Concurrent rendering - <https://react.dev/blog/2022/03/29/react-v18#what-is-concurrent-react>
13. Console logging - <https://developer.mozilla.org/en-US/docs/Web/API/console>
15. Context Provider - <https://react.dev/reference/react/createContext#provider>
17. Controlled component - <https://react.dev/reference/react-dom/components/input#controlling-an-input-with-a-state-variable>
19. Custom Hook - <https://react.dev/learn/reusing-logic-with-custom-hooks>
21. dangerouslySetInnerHTML - <https://react.dev/reference/react-dom/components/common#dangerously-setting-the-inner-html>
23. Default props - <https://react.dev/learn/passing-props-to-a-component#specifying-a-default-value-for-a-prop>
25. Derived state - <https://react.dev/learn/you-might-not-need-an-effect#updating-state-based-on-props-or-state>
27. DOM manipulation - <https://react.dev/learn/manipulating-the-dom-with-refs>
29. Effect with cleanup - <https://react.dev/reference/react/useEffect#connecting-to-an-external-system>
31. Event handler - <https://react.dev/learn/responding-to-events>
33. Event object - <https://react.dev/reference/react-dom/components/common#react-event-object>
35. fireEvent - <https://testing-library.com/docs/dom-testing-library/api-events>
37. Form Actions - <https://react.dev/reference/react-dom/components/form#handle-form-submission-with-an-action-prop>
39. Form submission - <https://react.dev/reference/react-dom/components/form#handle-form-submission-with-an-event-handler>
41. Functional Component - <https://react.dev/learn/your-first-component>
43. Higher-Order Component - <https://react.dev/reference/react/memo#usage>
45. Inline styling - <https://react.dev/reference/react-dom/components/common#applying-css-styles>
47. Jotai - <https://jotai.org/>
49. Key prop - <https://react.dev/learn/rendering-lists#keeping-list-items-in-order-with-key>
51. Lifting state up - <https://react.dev/learn/sharing-state-between-components>
53. List rendering - <https://react.dev/learn/rendering-lists>
55. Mock functions - <https://jestjs.io/docs/mock-functions>
57. Multiple inputs - <https://react.dev/learn/updating-objects-in-state>
59. NavLink - <https://reactrouter.com/api/components/NavLink>
61. Next.js - <https://nextjs.org/docs>
63. Passing arguments - <https://react.dev/learn/responding-to-events#passing-event-handlers-as-props>
65. Production build - <https://react.dev/learn/start-a-new-react-project#production-grade-react-frameworks>
67. Prop destructuring - https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Destructuring_assignment
69. Pure component - <https://react.dev/reference/react/memo>
71. React DevTools - <https://react.dev/learn/react-developer-tools>
73. Redux Toolkit - <https://redux-toolkit.js.org/>
75. Ref cleanup - <https://react.dev/blog/2024/12/05/react-19#cleanup-functions-for-refs>
77. Refs in loops - <https://react.dev/learn/manipulating-the-dom-with-refs#how-to-manage-a-list-of-refs-using-a-ref-callback>
10. componentDidCatch - <https://react.dev/reference/react/Component#componentdidcatch>
12. Conditional rendering - <https://react.dev/learn/conditional-rendering>
14. Context API - <https://react.dev/reference/react/createContext>
16. Context with reducer - <https://react.dev/learn/scaling-up-with-reducer-and-context>
18. Custom events - https://developer.mozilla.org/en-US/docs/Web/Events/Creating_and_triggering_events
20. Custom provider - <https://react.dev/learn/scaling-up-with-reducer-and-context#moving-all-wiring-into-a-single-file>
22. Data fetching - <https://react.dev/reference/react/useEffect#fetching-data-with-effects>
24. Dependency array - <https://react.dev/reference/react/useEffect#specifying-reactive-dependencies>
26. Document Metadata - <https://react.dev/reference/react-dom/components/title>
28. Effect synchronization - <https://react.dev/learn/synchronizing-with-effects>
30. Error Boundary - <https://react.dev/reference/react/Component#catching-rendering-errors-with-an-error-boundary>
32. Event listeners - <https://react.dev/learn/synchronizing-with-effects#subscribing-to-events>
34. File input - <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/input/file>
36. Focus management - <https://react.dev/learn/manipulating-the-dom-with-refs#how-to-manage-focus-using-a-ref>
38. Form labels - <https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements/label>
40. Fragment - <https://react.dev/reference/react/Fragment>
42. getDerivedStateFromError - <https://react.dev/reference/react/Component#static-getderivedstatefromerror>
44. Inline handler - <https://react.dev/learn/responding-to-events#adding-event-handlers>
46. Jest - <https://jestjs.io/docs/tutorial-react>
48. JSX expression - <https://react.dev/learn/javascript-in-jsx-with-curly-braces>
50. Keyboard navigation - https://developer.mozilla.org/en-US/docs/Web/Accessibility/Keyboard-navigable_JavaScript_widgets
52. Link - <https://reactrouter.com/api/components/Link>
54. Live regions - https://developer.mozilla.org/en-US/docs/Web/Accessibility/ARIA/Guides/Live_regions
56. Multiple contexts - <https://react.dev/learn/passing-data-deeply-with-context#using-and-providing-context-from-the-same-component>
58. Named handler - <https://react.dev/learn/responding-to-events#reading-props-in-event-handlers>
60. Nested routes - <https://reactrouter.com/start/declarative/routing>
62. Outlet - <https://reactrouter.com/api/components/Outlet>
64. Portal - <https://react.dev/reference/react-dom/createPortal>
66. Profiler - <https://react.dev/reference/react/Profiler>
68. Props - <https://react.dev/learn/passing-props-to-a-component>
70. React Compiler - <https://react.dev/learn/react-compiler>
72. React Testing Library - <https://testing-library.com/docs/react-testing-library/intro/>
74. Ref as prop - <https://react.dev/blog/2024/12/05/react-19#ref-as-a-prop>
76. Ref forwarding - <https://react.dev/reference/react/forwardRef>
78. render - <https://testing-library.com/docs/react-testing-library/api#render>

79. Render props - <https://react.dev/reference/react/cloneElement#passing-data-with-a-render-prop>
81. Routes - <https://reactrouter.com/api/components/Routes>
83. Select dropdown - <https://react.dev/reference/react-dom/components/select>
85. Server Actions - <https://react.dev/reference/rsc/use-server>
87. Snapshot testing - <https://jestjs.io/docs/snapshot-testing>
89. State batching - <https://react.dev/blog/2022/03/29/react-v18#new-feature-automatic-batching>
91. Storing values - <https://react.dev/reference/react/useRef#referencing-a-value-with-a-ref>
93. Strict Mode - <https://react.dev/reference/react/StrictMode>
95. Tailwind CSS - <https://tailwindcss.com/docs/utility-first>
97. TanStack Router - <https://tanstack.com/router/latest>
99. Textarea - <https://react.dev/reference/react-dom/components/textarea>
101. Turbopack - <https://nextjs.org/docs/app/api-reference/turbopack>
103. use hook - <https://react.dev/reference/react/use>
105. useCallback - <https://react.dev/reference/react/useCallback>
107. useDeferredValue - <https://react.dev/reference/react/useDeferredValue>
109. useEffectEvent - https://react.dev/reference/react/experimental_useEffectEvent
111. useId - <https://react.dev/reference/react/useId>
113. useLayoutEffect - <https://react.dev/reference/react/useLayoutEffect>
115. useMemo - <https://react.dev/reference/react/useMemo>
117. useOptimistic - <https://react.dev/reference/react/useOptimistic>
119. useReducer - <https://react.dev/reference/react/useReducer>
121. userEvent - <https://testing-library.com/docs/user-event/intro>
123. useTransition - <https://react.dev/reference/react/useTransition>
125. Vitest - <https://vitest.dev/>
80. Route - <https://reactrouter.com/api/components/Route>
82. screen queries - <https://testing-library.com/docs/queries/about>
84. Semantic HTML - https://developer.mozilla.org/en-US/docs/Glossary/Semantics#semantics_in_html
86. Server Components - <https://react.dev/reference/rsc/server-components>
88. startTransition - <https://react.dev/reference/react/startTransition>
90. State updater function - <https://react.dev/reference/react/useState#updating-state-based-on-the-previous-state>
92. Streaming SSR - <https://react.dev/reference/react-dom/server/renderToPipeableStream>
94. Suspense - <https://react.dev/reference/react/Suspense>
96. TanStack Query - <https://tanstack.com/query/latest/docs/framework/react/overview>
98. TanStack Start - <https://tanstack.com/start/latest>
100. Try-catch in handlers - <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/try...catch>
102. Uncontrolled component - <https://react.dev/reference/react-dom/components/input#providing-an-initial-value-for-an-input>
104. useActionState - <https://react.dev/reference/react/useActionState>
106. useContext - <https://react.dev/reference/react/useContext>
108. useEffect - <https://react.dev/reference/react/useEffect>
110. useFormStatus - <https://react.dev/reference/react-dom/hooks/useFormStatus>
112. useImperativeHandle - <https://react.dev/reference/react/useImperativeHandle>
114. useLocation - <https://reactrouter.com/api/hooks/useLocation>
116. useNavigate - <https://reactrouter.com/api/hooks/useNavigate>
118. useParams - <https://reactrouter.com/api/hooks/useParams>
120. useRef - <https://react.dev/reference/react/useRef>
122. useState - <https://react.dev/reference/react/useState>
124. ViewTransition - <https://react.dev/reference/react/ViewTransition>
126. waitFor - <https://testing-library.com/docs/dom-testing-library/api-async#waitFor>

Articles, guides, & named-author resources

127. Alt text - <https://www.w3.org/WAI/tutorials/images/>
129. Children typing - https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/function_components#react-component-props
131. Compound components - <https://kentcdodds.com/blog/compound-components-with-react-hooks>
133. Context performance - <https://kentcdodds.com/blog/how-to-optimize-your-context-value>
135. Create React App (legacy) - <https://create-react-app.dev/>
137. CSS-in-JS - <https://cssinjs.org/>
139. Error boundary library - <https://github.com/bvaughn/react-error-boundary>
141. Event delegation - <https://legacy.reactjs.org/blog/2020/08/10/react-v17-rc.html>
143. Form libraries - <https://react-hook-form.com/get-started>
145. Generic components - https://react-typescript-cheatsheet.netlify.app/docs/advanced/patterns_by_usecase#generic-components
147. List virtualization - <https://react-window.vercel.app/>
149. Props interface - https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/function_components
151. Recoil - <https://recoiljs.org/>
153. Remix - <https://remix.run/blog/merging-remix-and-react-router>
155. Skip links - <https://webaim.org/techniques/skipnav/>
157. State colocation - <https://kentcdodds.com/blog/state-colocation-will-make-your-react-app-faster>
159. SWR - <https://swr.vercel.app/>
128. Babel - <https://babeljs.io/docs/>
130. Color contrast - <https://www.w3.org/WAI/WCAG21/Understanding/contrast-minimum.html>
132. Container/Presentational - <https://www.patterns.dev/react/presentational-container-pattern/>
134. Context typing - <https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/context>
136. CSS Modules - <https://github.com/css-modules/css-modules>
138. Emotion - <https://emotion.sh/docs/introduction>
140. ESLint - <https://eslint.org/>
142. Event typing - https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/forms_and_events
144. Gatsby - <https://www.gatsbyjs.com/docs/glossary/static-site-generator/>
146. Interval/Timeout - <https://overreacted.io/making-setinterval-declarative-with-react-hooks/>
148. MobX - <https://mobx.js.org/>
150. React Native - <https://reactnative.dev/>
152. Ref typing - <https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/hooks#useref>
154. Sass/SCSS - <https://sass-lang.com/guide/>
156. Source maps - <https://webpack.js.org/configuration/devtool/>
158. Styled Components - <https://styled-components.com/docs/basics>
160. TypeScript - <https://www.typescriptlang.org/docs/handbook/2/basic-types.html>

161. useState typing - <https://react-typescript-cheatsheet.netlify.app/docs/basic/getting-started/hooks#usestate>

163. Webpack - <https://webpack.js.org/concepts/>

162. Vite - <https://vite.dev/guide/why>

164. Zustand - <https://zustand.docs.pmnd.rs/learn/getting-started/introduction>